

1 Physikalischer Kern: 2D-Strahlensegmentschnitt

Der rechnerische Kern der Physik-Engine basiert auf robuster linearer Algebra, um Randkollisionen zu lösen. Eine genaue mathematische Modellierung in diesem Stadium ist für hochauflösende akustische Simulationen entscheidend. Um den Rechendurchsatz zu maximieren, was bei der Berechnung von Millionen von Strahlen zwingend erforderlich ist, verzichtet die Engine auf rechnerisch aufwändige trigonometrische Funktionen zugunsten reiner Vektormathematik.

1.1 Geometrische Definitionen

Die beiden wichtigsten geometrischen Entitäten im 2D-Simulationsraum sind der akustische Strahl und die physikalische Grenze (die eine Wand darstellt).

Der Strahl Ein Schallstrahl wird durch einen Ursprungspunkt O und einen normalisierten Richtungsvektor definiert $\vec{D}$. Jeder Punkt P entlang der Strahlenbahn kann als Funktion der skalaren Distanz t entlang dieser Richtung berechnet werden:

$$P(t) = O + t\vec{D}$$

wobei $t \geq 0$ sicherstellt, dass der Strahl vom Emitter aus im Raum weitergeht.

Die Wand (Die Strecke) Eine Wand wird als Liniensegment dargestellt, das durch zwei Endpunkte, A und B , definiert ist. Jeder Punkt W auf diesem Segment wird durch Interpolation zwischen den Endpunkten mit einem skalaren Parameter u gefunden. Definiert man den Wandvektor als $\vec{V} = B - A$, ergibt sich die parametrische Gleichung durch:

$$W(u) = A + u\vec{V}$$

Damit ein Punkt strikt innerhalb der physikalischen Grenzen des Wandsegments liegt, muss die Bedingung $0 \leq u \leq 1$ erfüllt sein.

1.2 Schnittpunktberechnung

Um den Schnittpunkt zu bestimmen, werden die Strahlen- und Wandgleichungen gleichgesetzt:

$$O + t\vec{D} = A + u\vec{V}$$

Dies bildet ein System linearer Gleichungen, um t (Ausbreitungsdistanz) und u (Segmentschnittparameter) zu lösen. Durch die Definition eines Vektors $\vec{E} = A - O$, der vom Ursprung des Strahls zum Ausgangspunkt der Wand zeigt, ordnet sich die Gleichung um:

$$t\vec{D} - u\vec{V} = \vec{E}$$

Im zweidimensionalen Raum kann dieses System effizient mit dem 2D-Analogen des Kreuzprodukts gelöst werden (oft als Perp-Dot-Produkt bezeichnet). Für

zwei beliebige 2D-Vektoren $\vec{v}_1 = (x_1, y_1)$ und $\vec{v}_2 = (x_2, y_2)$ ergibt sich eine skalare Determinante: $\vec{v}_1 \times \vec{v}_2 = x_1 y_2 - y_1 x_2$.

Durch die Definition der Schnittdeterminante als $\det = \vec{D} \times \vec{V}$ können die Parameter t und u isoliert und algebraisch gelöst werden:

$$t = \frac{\vec{E} \times \vec{V}}{\det}$$

$$u = \frac{\vec{E} \times \vec{D}}{\det}$$

1.3 Schnittpunkt-Validierung (Trefferbedingungen)

Eine mathematisch gültige Lösung des linearen Systems garantiert nicht automatisch eine physikalische Kollision. Die folgenden Einschränkungen müssen überprüft werden, um eine gültige akustische Reflexion zu bestätigen:

- **Parallelität:** Wenn $\det \approx 0$, sind der Strahl und die Wand perfekt parallel und schneiden sich niemals. In der Implementierung ist ein kleiner Epsilon-Schwellenwert anstelle einer exakten Nullprüfung verwendet, um Gleitkommaraunauigkeiten zu vermeiden:

```
if det.abs() < 1e-6 {
    return None;
}
```

- **Richtungsabhängigkeit:** Wenn $t < 0$, schneiden sich die mathematischen Geraden, aber die Kollision erfolgt hinter dem Aussendungspunkt des Strahls:

```
if ray_t < 0.0 {
    return None;
}
```

- **Segmentgrenzen:** Wenn $u < 0$ oder $u > 1$, schneidet der Strahl zwar die unendliche Gerade, auf der die Wand liegt, verfehlt jedoch die physische Strecke

```
if wall_u < 0.0 || wall_u > 1.0 {
    return None;
}
```

1.4 Vektorreflexion

Nach der Bestätigung eines gültigen Schnittpunkts muss die resultierende Trajektorie der Schallwelle berechnet werden. Dies erfordert den Oberflächennormalenvektor

$\vec{N}$, der streng senkrecht von der Wand weg zeigt Für einen gegebenen 2D-Wandvektor $\vec{V} = (x, y)$ ist die normierte senkrechte Normale $\vec{N} = (-y, x)$.

Die Richtung des reflektierten Strahls $\vec{R}$ wird bestimmt, indem von der Richtung des einfallenden Strahls $\vec{D}$ das Doppelte der Projektion dieser Richtung auf die Oberflächennormale subtrahiert wird.

$$\vec{R} = \vec{D} - 2(\vec{D} \cdot \vec{N})\vec{N}$$

Innerhalb der Rust-Codebasis wird diese Berechnung der Vektorreflexion durch die `reflect` Methode optimiert, die vom `glam`-Crate bereitgestellt wird und Single Instruction, Multiple Data (SIMD)-Instruktionen nutzt, um diese Operationen mit minimalem Overhead durchzuführen.

2 Hochleistungs-Nebenläufigkeit und Datenparallelität

Ein wesentlicher Vorteil von Rust im High-Performance-Computing sind seine strengen Garantien zur Kompilierzeit gegen Data-Races. Die Verteilung einer rechenintensiven, iterativen Physikscheife auf Multi-Core-Prozessorarchitekturen erfordert in traditionellen Programmiersprachen oft komplexe Thread-Sperrlogiken (z.B. Mutexe), um Speicherbeschädigungen zu verhindern. In dieser Engine wird dieser Overhead vermieden durch ein funktionales, datenparalleles Paradigma unter Verwendung des `rayon`-Crates.

2.1 Architektonischer Wandel: Von Shared Mutation zu thread-lokalem-Folding

Bei einer standardmäßigen Single-Thread-Ausführung hängen sequenzielle Iterationen die berechneten Schnittpunktdate (z.B. `delay` und `pressure` werte) an ein gemeinsames dynamisches Array an. Diese Schleife auf zahlreiche Threads aufzuteilen, würde zur gleichzeitigen Schreibversuchen auf dieselben Speicheradressen führen, was unmittelbare Data Races zur Folge hätte. Anstatt diese gemeinsam genutzt Arrays in rechenintensive, threadsichere Sperren (Locks) zu hüllen, nutzt die Architektur das Trait `into_par_iter()` von `rayon` in Kombination mit einer `fold`- und `reduce`- Pipeline.

Dieser Ansatz teilt die Arbeitslast sicher auf alle verfügbaren CPU-Kerne auf. Anstatt zu versuchen, in ein globales Array zu schreiben oder Speicher für jeden Strahl dynamisch zu allokalieren, verwendet die Engine die `fold`-Operation, um einen privaten, vorab allokalieren, thread-lokalen Puffer für jeden Kern bereitzustellen.

Jeder Thread simuliert unabhängig seine zugewiesene Teilmenge an Strahlen und hängt die Schnittpunktergebnisse sequenziell in seinen eigenen lokalen Speicherbereich an, ohne Sperren (lock-free). Sobald ein Thread seine Arbeitslast abgeschlossen hat, wird die `reduce`-Phase aktiviert, welche diese isolierten thread-lokalen Puffer sicher und effizient zu den endgültigen, vereinheitlichten

Arrays zusammenführt. Dieser funktionale Ansatz garantiert die Datensicherheit und maximiert gleichzeitig die Hardwareauslastung.

```
use rayon::prelude::*;

fn run_simulation_parallel(
    config: &SimulationConfig,
    walls: &Vec<Wall>
) -> (Vec<f32>, Vec<f32>) {
    let (delays_par, pressures_par, _) = (0..config.rays_to_cast)
        .into_par_iter()
        // 1. Falten, Split
        .fold(|| {
            let local_delays = Vec::with_capacity(CAPACITY);
            let local_pressures = Vec::with_capacity(CAPACITY);
            let temp_ray_buffer = Vec::with_capacity(CAP);

            (local_delays, local_pressures, temp_ray_buffer)
        },
        |mut local_buf, _| {
            let fresh_ray = generate_initial_ray(config);
            simulate_single_ray(fresh_ray, walls, config, &mut local_buf.2);
            for (delay, pressure) in &local_buf.2 {
                local_buf.0.push(*delay);
                local_buf.1.push(*pressure);
            }
            local_buf
        })
        // 2. Reduzieren, Global Merge
        .reduce(
            || (Vec::new(), Vec::new(), Vec::new()),
            |mut a, mut b| {
                a.0.append(&mut b.0);
                a.1.append(&mut b.1);
                a
            }
        );
    (delays_par, pressures_par)
}
```

3 Leistungsoptimierungen und Profiling

Benchmarking the compiled binary (utilizing Rust's `--release` flag for maximum LLVM optimization) revealed significant bottlenecks and subsequent optimization opportunities within the parallelized architecture.

Das Benchmarking der kompilierten Binärdatei (unter Verwendung des `--release` flag und dem `let var = Instant::now(); timer`) deckte signifikante Engpässe und darauffolgende Optimierungsmöglichkeiten innerhalb der parallelisierten Architektur auf.

3.1 Reduzierung des Speicherallokations-Overheads: `flat_map` vs. `fold`

Während die anfängliche parallele Implementierung den `flat_map`-iterator von `rayon` nutzt, offenbarte die Skalierung der Simulation auf 100.000.000 Strahlen einen signifikanten Leistungsengpass bezüglich der Speicherallokation. Die `flat_map`-Operation allozierte dynamisch einen neuen `Vec` für die Rückgabedaten jedes einzelnen Strahls. Bei der Verteilung auf alle CPU-Kerne führte dies zu Millionen von flüchtigen Heap-Allokationen pro Sekunde. Das System erlebte Memory Thrashing; die CPU-Auslastung fiel auf 0% der Hauptspeicher war ausgelastet, und das Betriebssystem entzog dem Ausführungs-Thread die Ressourcen, was zum Stillstand der Simulation führte.

Um diesen Allokations-Overhead zu beheben, wurde die parallele Pipeline refaktoriert, um das `fold`-Muster zu verwenden. Anstatt Speicher pro Strahl zu allokalieren, stellt die `fold`-Operation exakt einen thread-lokalen Puffer (initialisiert über `Vec::with_capacity();`) pro CPU-Kern bereit. Die Kerne akkumulieren ihre lokalen Ergebnisse und führen diese erst nach Abschluss des Threads zusammen. Diese architektonische Änderung reduzierte die Heap-Allokationen um mehrere Größenordnungen und ermöglichte es dem System, 100.000.000 Strahlen effizient zu verarbeiten, ohne die Speicherressourcen zu erschöpfen.

3.2 Benchmarking Ergebnisse und I/O Overhead

Empirische Tests verdeutlichen die Wirksamkeit der optimierten Parallelisierung sowie die verborgenen Kosten von System-I/O und Thread-Initialisierung.

- **Paralleler Speedup:** Bei einer moderaten Arbeitslast wurde die Single-Thread-Simulation in 11,79 Sekunden abgeschlossen. Die durch `rayon` optimierte parallele Simulation bewältigt dieselbe Arbeitslast in 2,56 Sekunden, was einen $4,60\times$ Speedup auf der Test-Hardware ergab.
- **Thread-Pool-Kaltstart:** Bei extrem kleinen Arbeitslasten wies `rayon` einen Initialisierung-Overhead auf. In einem Benchmark mit wenigen Strahlen wurde die Single-Thread-Ausführung in 0,14 Sekunden abgeschlossen, während die parallele Ausführung 0,34 Sekunden in Anspruch nahm. Dies deutet darauf hin, dass die Kosten für die Initialisierung des Thread-Pools die Recheneinsparungen bei trivialen Arbeitslasten überwiegen.
- **The I/O Engpass:** Das Profiling der End-to-End-Ausführungszeit deckte einen massiven Engpass bei der Datenserialisierung auf. In einem Test mit 10.000.000 Strahlen wurde die interne Simulation schnell ausgeführt (4,72s Single-Thread vs. 0,64s Parallel). Die anschließende Serialisierung und

das Schreiben des Datensatzes dominierten jedoch die End-to-End-Laufzeit (37, 40s vs. 33, 24s), was die Notwendigkeit eines RAM-gestützten **BufWriter** unterstreicht.